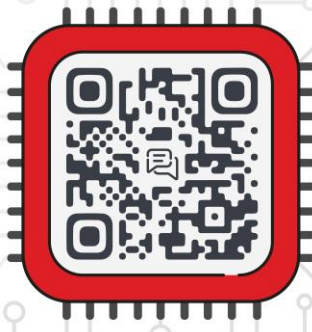




TD

6



4



6

sp

كلية الهندسة المعلوماتية

السنة الأولى

الوراثة (Inheritance)

د. ماهر صارم



RB Informatix; 14/5/2026

محتوى مجاني غير مخصص للبيع التجاري

البرمجة 2

الوراثة (Inheritance)

الوراثة هي آلية تسمح لكلاس جديد بأن يرث المتغيرات والتتابع من كلاس آخر قائم.

■ الكلاس الاب (super class):

هو الكلاس الأساسي الذي يحوي على الصفات والوظائف العامة.

■ الكلاس الابن (sub class):

هو الكلاس الذي يرث من الكلاس الأب ويمكنه إضافة صفات موجودة أو تعديل الصفات الموروثة.

مثال من الواقع:

لنفترض أننا نقوم ببناء برنامج لإدارة الموظفين في شركة ما. في هذا النظام، سنجد نوعين من الصفات والوظائف:

1. الصفات والوظائف العامة (الكلاس الأب - Super Class):

هناك بيانات مشتركة وموحدة يمتلكها جميع الموظفين في الشركة بغض النظر عن طبيعة عملهم، مثل:

الصفات (Attributes): (الاسم، الراتب).

الوظائف (Methods): (إجراء تسجيل الدخول إلى النظام).

بدلاً من تكرار كتابة هذه الكود لكل موظف، نجعلها في كلاس أساسي عام نسميه كلاس الموظف (Employee).

2. الصفات والوظائف الخاصة (الكلاسات الأبناء - Sub Classes):

تأتي الوراثة لتسمح لنا بإنشاء كلاسات فرعية تخص تخصصات معينة، ترث الميزات العامة وتضيف عليها ميزات

الخاصة، مثلاً:



كلاس المبرمج:

يرث تلقائياً من الكلاس الأب: (الاسم، الراتب، ودالة تسجيل الدخول).

يضيف صفة خاصة به فقط: (لغة البرمجة التي يعمل بها).

كلاس موظف من نوع آخر:

يرث تلقائياً من الكلاس الأب: (الاسم، الراتب، ودالة تسجيل الدخول).

يضيف صفة خاصة به تختلف عن المهندس..

ملاحظة:

في كلاس الأب يفضل استعمال protected بدلاً من private لأن protected تسمح فقط لكلاسات الأبناء بالتعديل على متغيرات الأب بالإضافة لأي كلاس يتواجد في نفس الحزمة، أما الكلاسات الأخرى لن تستطيع الوصول لكلاس الأب.

مثال لتوضيح الفكرة:

حساب بنكي لمستخدم (Customer) وحساب (Saving):

```
public class Customer {
    protected double balance;
    protected int accountNumber;
    protected String name;
    Customer (int accountNumber,String name){
        this.accountNumber=accountNumber;
        this.name=name; }
    void deposit(double money){
        balance+=money;
    }
    void withdraw(double money){
        if(money>balance){
            System.out.println("you do not have enough money
:( ");
        }
        else
            balance-=money;
    }
    void printInfo(){
        System.out.println("Account Number is: "+accountNumber);
        System.out.println("Customer name is: " +name);
        System.out.println("Balance = "+balance);
    }
}
```

```
public class Saving extends Customer{
    private double rate;

    Saving(int accountNumber, String name, double rate){
        super(accountNumber,name);
        this.rate=rate;
    }
    void CalculateRate(){
        balance=balance+(balance*rate);
    }
}
```

```
public class Main {
    public static void main (String[] args)
    {
        Saving S=new Saving(123,"Majd",0.12);
        S.deposit(512.20);
        S.printInfo();
        S.withdraw(12.20);
        S.printInfo();
        S.CalculateRate();
        S.printInfo();
    }
}
```

توضيح للكود السابق:

يتألف الكود من 3 كلاسات:

1- كلاس الأب (Customer): يُمثل العميل العام وحسابه البنكي.

هو كلاس عميل يعبر عن معلومات العميل، وفيه ثلاث توابع:

تابع (deposit): وهو تابع لإيداع المال في الحساب.

تابع (withdraw): وهو تابع لسحب المال من الحساب، لكن أولاً يتحقق ما إذا كان الحساب يحوي مالاً كافياً.

تابع طباعة معلومات.



2- كلاس الابن (Saving): يُمثل نوعاً خاصاً من الحسابات وهو "حساب التوفير" الذي يرث من العميل ويضيف ميزة الفائدة، يعبر عن حساب ادخار المال في البنك، يحتوي على متغير rate يعبر عن نسبة الربح التي يحصل عليها المستخدم.

3- كلاس التشغيل (Main): لتجربة الكود ورؤية النتائج.

نلاحظ عند استخدام الباني في السطر الرابع استعملنا super وهو يعبر عن الأب ونمرر داخله المتغيرات التي كانت موجودة في كلاس الأب.

ملاحظة: عند استخدام super يجب إمرار المتغيرات ضمنه بنفس الترتيب في كلاس الأب وإلا سيحصل خطأ.

super (accountNumber , name); —————> صحيحة

super (name, accountNumber); —————> خاطئة.

ويوجد ضمن كلاس saving تابع لحساب المبلغ كاملاً مع الفائدة.

نلاحظ، أن الرصيد balance لم يتم وضعه في الباني، وبالتالي قيمته الافتراضية ستبدأ من 0.

انتهت المحاضرة السادسة (٤٦)

Only dead fish go with
the flow.

